Implementation of Blocked QR Factorization on CPU Multi-threading

Wayne Anderson

14/07/2025

1. Introduction

QR factorization is widely used in cases like linear solver and finding eigenvalues and eigenvectors. The traditional approach for the QR algorithm involves column-by-column manipulation and followed by its corresponding matrix multiplication [1]. This article introduces a technique that applies QR decomposition block by block [2], with each block being able to fit in different memory hierarchies (like cache and shared memory).

Additionally, based on the blocked algorithm, this article conceives some methods to simplify matrix-matrix multiplications in the algorithm, reducing computing loads, which is proof to be feasible by the codes provided in the appendix.

2. Mathematical Description

The process of blocked QR factorization can be expressed as following. Given an m by n source matrix $\boldsymbol{A}$, is sliced to blocks with shape as m by b, assuming that

$$K = \frac{n}{b}, K \in N^{+} \tag{2.1}$$

The layout of i^{th} block $\boldsymbol{B}_k$ is shown below:

$$\mathbf{A} = \begin{pmatrix} \ddots & \cdots & \cdots & \cdots \\ \cdots & \mathbf{B}_{k-1} & (\cdots_{b\times b}) & (\cdots_{b\times(n-kb)}) \\ & & \mathbf{B}_k & \\ & & & \mathbf{A}_i \end{pmatrix} \tag{2.2}$$

$\boldsymbol{A}_i$ denotes the bottom-right submatrix for the i^{th} block. For each block, perform naive Householder reflection for each column.

A similar layout can be seen in the distribution of the k^{th} column (Eq. 2.3). V_k is the column vector of the block **B**, and then is applied with Householder transform. A Householder reflector $\boldsymbol{H}$ aims to transform a given linear system so that a specific vector w denoted in that linear space can be transformed to a new vector v.

$$\mathbf{B}_i = \begin{pmatrix} \ddots & \cdots & \cdots & \cdots \\ \cdots & \vec{V}_{k-1} & b_{k-1,k} & (\cdots_{1\times(n-kb)}) \\ & & \vec{V}_k & \\ & & & \mathbf{B}_k \end{pmatrix} \tag{2.3}$$

For its application on QR decomposition, involving eliminating the lower triangular part of a matrix, the new vector v will have such form:

$$\vec{v} = (v_1, 0, \ldots 0) \tag{2.4}$$

Hence, $\boldsymbol{H}$ can be expressed as:

$$\mathbf{H} = \mathbf{I} - 2\vec{w}\vec{w}^{T} \tag{2.5}$$

$$\vec{w} = \vec{v} - \vec{u} \tag{2.6}$$

In practice, the vectors should be normalized. The mathematical approach of Householder transform on one column is hence given, (note: the procedures are listed in meaningful order, please follow the order shown below (Eq. 2.7-2.11) to make sure the proper data is overwritten, such as v_2):

$$\partial = -sign(v_0) \tag{2.7}$$

$$v_2 = v_1 - \partial \,||\, \vec{V_k} \,||\, \tag{2.8}$$

$$\vec{V_k} = \frac{\vec{V_k}}{||\, \vec{V_k} \,||\,} \tag{2.9}$$

$$\mathbf{H} = \mathbf{I} - 2\vec{V_k}\vec{V_k}^{T} \tag{2.10}$$

$$\mathbf{B}_k = \mathbf{H}\mathbf{B}_k \tag{2.11}$$

Where v_k denotes the k^{th} element of $\vec{V_k}$. For traditional QR factorization algorithm, such procedure is taken column by column, involving lots of matrix multiplications.

However, the blocked algorithm aggregates some Householder reflectors and update the rest of the matrix by just one matrix multiply:

$$\mathbf{A}_i = \left(\prod_{k=1}^{b} \mathbf{H}_k \right) \mathbf{A}_{i-1} \tag{2.12}$$

The following steps show how the aggregation is processed. Matrix $\boldsymbol{W}$ and $\boldsymbol{Y}$ (both have the same sizes as that of the block) are introduced in the iterations within a block with b columns,

$$\mathbf{Y}(k) = \overrightarrow{V_k} \tag{2.13}$$

For the first column ($k = 1$),

$$\mathbf{W}(k) = 2Y(k) \tag{2.14}$$

For the rest of the columns,

$$\mathbf{W}(k) = 2\left(\mathbf{I} - \mathbf{W}(1:k-1)\mathbf{Y}(1:k-1)^T\right)\mathbf{Y}(k) \tag{2.15}$$

After this block is done processing, apply such transform to update the bottom-right submatrix of $\boldsymbol{A}$:

$$\mathbf{A}_k(k:,k:) = \left(\mathbf{I} - \mathbf{W}\mathbf{Y}^T\right)\mathbf{A}_{k-1}(k:,k:) \tag{2.16}$$

In this algorithm, matrix **Q**, an elementary initially, is also updated each time a block is processed, and can be obtained first when the entire process is done, unlike traditional QR decomposition, where matrix **R** is calculated earlier instead. The update procedure for **Q** is similar with matrix **A:**

$$\mathbf{Q}_k(:,k:) = \left(\mathbf{I} - \mathbf{W}\mathbf{Y}^T\right)\mathbf{Q}_{k-1}(:,k:) \tag{2.17}$$

Once the algorithm is finished processing entire matrix **A**, **Q** is also calculated. Matrix **R** can be obtained by:

$$\mathbf{R} = \mathbf{Q}^{-1}\mathbf{A}_0 \tag{2.18}$$

Where $\boldsymbol{A_0}$ denotes the original matrix **A**.

3. Simplification

3.1 Simplified update procedures for W

Even if the blocking strategy utilizes memory throughputs in machines with memory hierarchy, the whole process is rich in matrix-matrix multiplications, especially obtaining matrix **W** described in Eq. 2.15. As iteration goes on, k increases, the depth of matrices increased for term

$$\mathbf{I} - \mathbf{W}(1:k-1)\mathbf{Y}(1:k-1)^T \tag{3.1}$$

, resulting more computational loading. However, this term can be simplified, and guaranteed only one vector-vector outer product, followed by one matrix-matrix elementwise subtraction, by the following techniques:

Let's break down the iteration step by step, for k=1,

$$\mathbf{W}_1 = 2\overrightarrow{V_1} \tag{3.2}$$

for k=2,

$$\mathbf{W}_2 = 2(\mathbf{I} - \mathbf{W}_1\overrightarrow{V_1}^T)\overrightarrow{V_2} \tag{3.3}$$

for k=3,

$$\begin{aligned} \mathbf{W}_3 &= 2\left(\mathbf{I} - \mathbf{W}_{1:2}\mathbf{Y}_{1:2}{}^T\right)\overrightarrow{V_3} \\ \mathbf{W}_3 &= 2\left[\mathbf{I} - \left(\mathbf{W}_1\overrightarrow{V_1}^T + \mathbf{W}_2\overrightarrow{V_2}^T\right)\right]\overrightarrow{V_3} \\ \mathbf{W}_3 &= 2\left(\mathbf{I} - \mathbf{W}_1\overrightarrow{V_1}^T - \mathbf{W}_2\overrightarrow{V_2}^T\right)\overrightarrow{V_3} \end{aligned} \tag{3.4}$$

for general case, considering Eq. 2.13 and Eq. 2.15, matrix **W** and **Y** are assembled column by column, the following equation can be deduced:

$$\mathbf{W}_k = 2\left[\mathbf{I} - \sum_{i=1}^{k} \mathbf{W}_i\overrightarrow{V_i}^T\right]\overrightarrow{V_3} \tag{3.5}$$

Hence, a n-by-n matrix can be defined to hold the transients and update itself at each iteration. In implementation of DECX, this matrix is called $\boldsymbol{I_{wy}}$. The process of obtaining matrix **W** can be simplified as:

$$\begin{aligned} \mathbf{I}_{wy}|_k &= \begin{cases} \mathbf{I} & ,k=1 \\ \mathbf{I}_{wy}|_{k-1} - \mathbf{W}_k\overrightarrow{V_k}^T & ,otherwise \end{cases} \\ \mathbf{W}_k &= \left(\mathbf{I}_{wy}|_k\right)\overrightarrow{V_k} \end{aligned} \tag{3.6}$$

3.2 Simplified update procedures for A and Q

Recall on Eq. 2.16 and Eq. 2.17, **A** and **Q** are both updated after the block process is finished. A similar structure in the updater can be seen, $\boldsymbol{I - WY^T}$. The accumulated matrix $\boldsymbol{I_{wy}}$ can be used. Recall on Eq. 3.5, when the block process is done, the value of $\boldsymbol{I_{wy}}$ is

$$\mathbf{I}_{wy} = \mathbf{I} - \sum_{i=1}^{b-1} \mathbf{W}_i\overrightarrow{V_i}^T \tag{3.7}$$

Recall on Eq. 3.5, we have:

$$\begin{aligned} \mathbf{I} - \mathbf{W}\mathbf{Y}^T &= \mathbf{I} - \sum_{i=1}^{b-1} \mathbf{W}_i\overrightarrow{V_i}^T - \mathbf{W}_b\overrightarrow{V_b}^T \\ &= \mathbf{I}_{wy}|_{k=b} - \mathbf{W}_b\overrightarrow{V_b}^T \end{aligned} \tag{3.8}$$

Hence, only one vector-vector outer product is needed to obtain the updater.

The implementation code is provided in appendix 5.2.

3.3 Simplification for calculating **R**

Considering the characteristic of QR decomposition, matrix **Q** is always orthogonal, thus:

$$\mathbf{R} = \mathbf{Q}^{-1}\mathbf{A}_0 = \mathbf{Q}^T\mathbf{A}_0 \tag{3.9}$$

There is no need to calculate the inverse of matrix **Q**.

4. Concurrency

there are more than one CPUs in modern computational architectures. Both the block process and the block level updating can be broken down into concurrent processes.

4.1 Concurrency within block process

According to the block QR algorithm, a naïve serial process can be described as:

```
for i = 1:b
   vi = block(:, i);
   vi_post = HouseholderOneCol(vi);
   H = I – 2*vi_post*vi_post.';
   block(i, i:) = UpdateBlock(H);
   W(i) = UpdateW(vi_post);
end
```

By identifying the dependency, the concurrent version of block process can be expressed as the block diagram (see Figure 4.1).

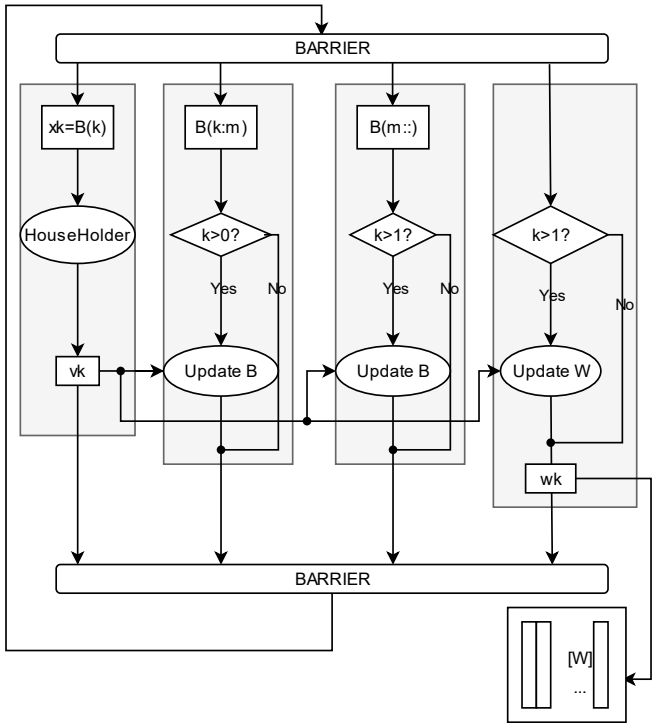

Figure 4.1 Concurrency within block process

Where m is an integer between k and b, usually tuned and defined in runtime based on compute load balancing.

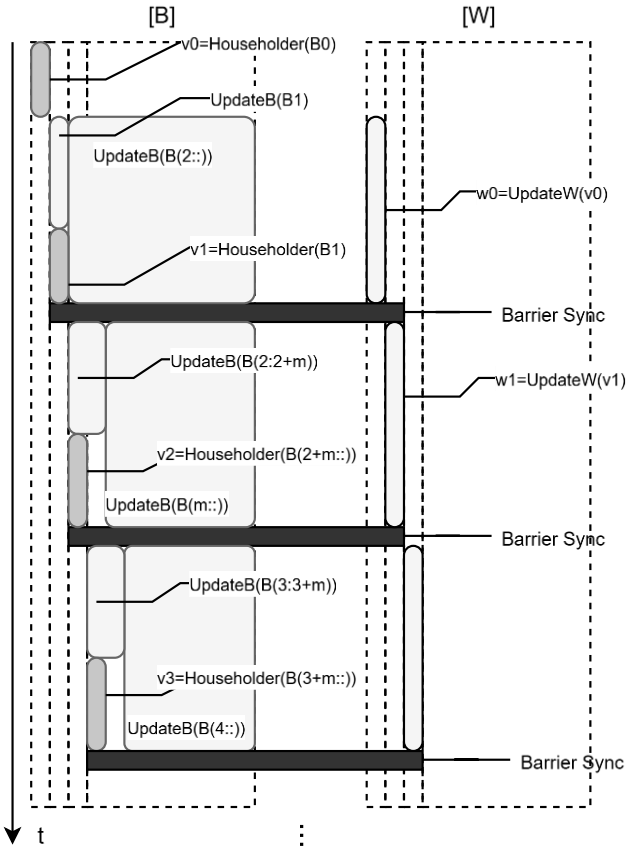

Figure 4.2 Timing diagram of concurrent block process

4.2 Concurrency for block-level process

5. References

[1] J. Lybbert, F. Li, "QR Decomposition Algorithms," 2024.

[2] F. Rotella and I. Zambettakis, "Block Householder Transformation for Parallel QR Factorization," Applied Mathematics Letters, vol. 12, no. 4, pp. 29-34, 1999.

Appendix

5.1 MatLab codes for prototype of blocked QR factorization

```
A_6x6 = [10 20 30 40 50 60;
         32 32 44 55 66 35;
         23 66 74 64 45 65;
         67 28 46 26 46 42;
         95 95 52 88 65 11;
         75 53 96 47 32 32];
A = zeros(size(A_6x6));
A(:, :) = A_6x6(:, :);
[Q, R] = Block_QR_Process_HH(A);
[Q_ref, R_ref] = qr(A_6x6);

function [V_ret, beta_ret] = QR_block_householder(panel)
    [v_len, block_dim] = size(panel);
    V = zeros(v_len, block_dim);
    beta = zeros(1, block_dim);
    computed_num = 0;
    for k = 1:block_dim
        if (v_len == block_dim && k == block_dim)
            continue;
        end
        vk = panel(k:v_len, k);
        if (sum(abs(vk - zeros(size(vk)))) < 1e-9)
            disp("skip when vk is all zero");
            continue;
        end
        x0 = vk(1);
        vk_norm2 = norm(vk, 2);
        sign = -1;
        if (x0 < 0)
            sign = 1;
        end
        vk(1) = x0 - sign * vk_norm2;
        vk_norm2_ = norm(vk, 2);
        vk = vk ./ vk_norm2_;
        beta(k) = 2.0;
        % Update rest of the pannel
        H = eye(v_len - k + 1) - 2 .* (vk * vk.');
        pannel_R = H * panel(k:v_len, k:block_dim);
        panel(k:v_len, k:block_dim) = pannel_R;
        V(k:v_len, k) = vk;
        computed_num = computed_num + 1;
    end
    V_ret = V(:, 1:computed_num);
    beta_ret = beta(1:computed_num);
end

function [W, Y] = compute_WY(V, beta)
    [v_len, block_dim] = size(V);
    W = zeros(v_len, block_dim);
    Y = zeros(v_len, block_dim);
    Y(:, 1) = V(:, 1);
    W(:, 1) = beta(1) .* V(:, 1);
    for i = 2 : block_dim
        z = beta(i) .* (eye(v_len) - W(:, 1:i-1) * (Y(:, 1:i-1)).') * V(:, i);
        W(:, i) = z;
        Y(:, i) = V(:, i);
    end
end

function [Q, R] = Block_QR_Process_HH(mat)
    [h, w] = size(mat);
    if (h ~= w)
        disp("The input matrix must be a square");
        return;
    end
    mat_copy = zeros(size(mat));
    mat_copy(:, :) = mat(:, :);
    N = w;
    block_dim = 3;
    Q = eye(N);
    for i=1:block_dim:N
        block_cols = mat(i : N, i : i+block_dim-1);
        [V, beta] = QR_block_householder(block_cols);
        [W, Y] = compute_WY(V, beta);
        IWY = eye(N - i + 1) - W * Y.';
        mat_R = mat(i : N, i : N);
        mat(i : N, i : N) = IWY.' * mat_R;
        % Notice: Update all rows of Q each time
        Q_Rcols = Q(:, i : N);
        Q_Rcols = Q_Rcols * IWY;
        Q(:, i : N) = Q_Rcols;
    end
    % Q is orthogonal, hence Q^T = Q^-1
    R = Q.' * mat_copy;
end
```

5.2 MatLab codes for simplifying computation

```
A_6x6 = [10 20 30 40 50 60;
         32 32 44 55 66 35;
         23 66 74 64 45 65;
         67 28 46 26 46 42;
         95 95 52 88 65 11;
         75 53 96 47 32 32];
A = zeros(size(A_6x6));
A(:, :) = A_6x6(:, :);
[Q, R] = Block_QR_Process_HH(A);
[Q_ref, R_ref] = qr(A_6x6);
function [V_ret, W_ret, IWY] = QR_block_householder(panel)
  [v_len, block_dim] = size(panel);
  V = zeros(v_len, block_dim);
  W = zeros(v_len, block_dim);
  beta = zeros(1, block_dim);
  computed_num = 0;
  IWY = eye(v_len);
  for k = 1:block_dim
    if (v_len == block_dim && k == block_dim)
      continue;
    end
    vk = panel(k:v_len, k);
    x0 = vk(1);
    vk_norm2 = norm(vk, 2);
    sign = -1;
    if (x0 < 0)
      sign = 1;
    end
    vk(1) = x0 - sign * vk_norm2;
    vk_norm2_ = norm(vk, 2);
    vk = vk ./ vk_norm2_;
    beta(k) = 2.0;
    V(k:v_len, k) = vk;
    if (k == 1)
      W(:, 1) = beta(1) .* V(:, 1);
    else
      IWY = IWY - W(:, k-1) * (V(:, k-1)).';
      z = beta(k) .* IWY * V(:, k);
      W(:, k) = z;
    end
    if (k < block_dim)
    % Update rest of the pannel
    H = 2.* (vk * vk.') * panel(k:v_len, k+1:block_dim);
    panel(k:v_len, k+1:block_dim) = panel(k:v_len, k+1:block_dim)- H;
    end
     computed_num = computed_num + 1;
  end
  V_ret = V(:, 1:computed_num);
  W_ret = W(:, 1:computed_num);
end

function [Q, R] = Block_QR_Process_HH(mat)
    [h, w] = size(mat);
    if (h ~= w)
        disp("The input matrix must be a square");
        return;
    end
    mat_copy = zeros(size(mat));
    mat_copy(:, :) = mat(:, :);
    N = w;
    block_dim = 3;
    Q = eye(N);
    for i=1:block_dim:N
        block_cols = mat(i : N, i : i+block_dim-1);
        [V, W, IWY] = QR_block_householder(block_cols);
        [h, w] = size(W);
        IWY = IWY - W(:, w) * V(:, w).';
        % IWY = eye(N - i + 1) - W * V.';
        mat_R = mat(i : N, i : N);
        mat(i : N, i : N) = IWY.' * mat_R;
        % Notice: Update all rows of Q each time
        Q_Rcols = Q(:, i : N);
        Q_Rcols = Q_Rcols * IWY;
        Q(:, i : N) = Q_Rcols;
    end
    % Q is orthogonal, hence Q^T = Q^-1
    R = Q.' * mat_copy;
end
```